

FIAP · PÓS TECH · TECH CHALLENGE — FASE 4

Mecânica API

Arquitetura de Microserviços & Saga

Refatoração para microserviços independentes, com Saga Pattern, persistência poliglota, IaC (Terraform) e CI/CD em Kubernetes (EKS).

Grupo 14SOAT

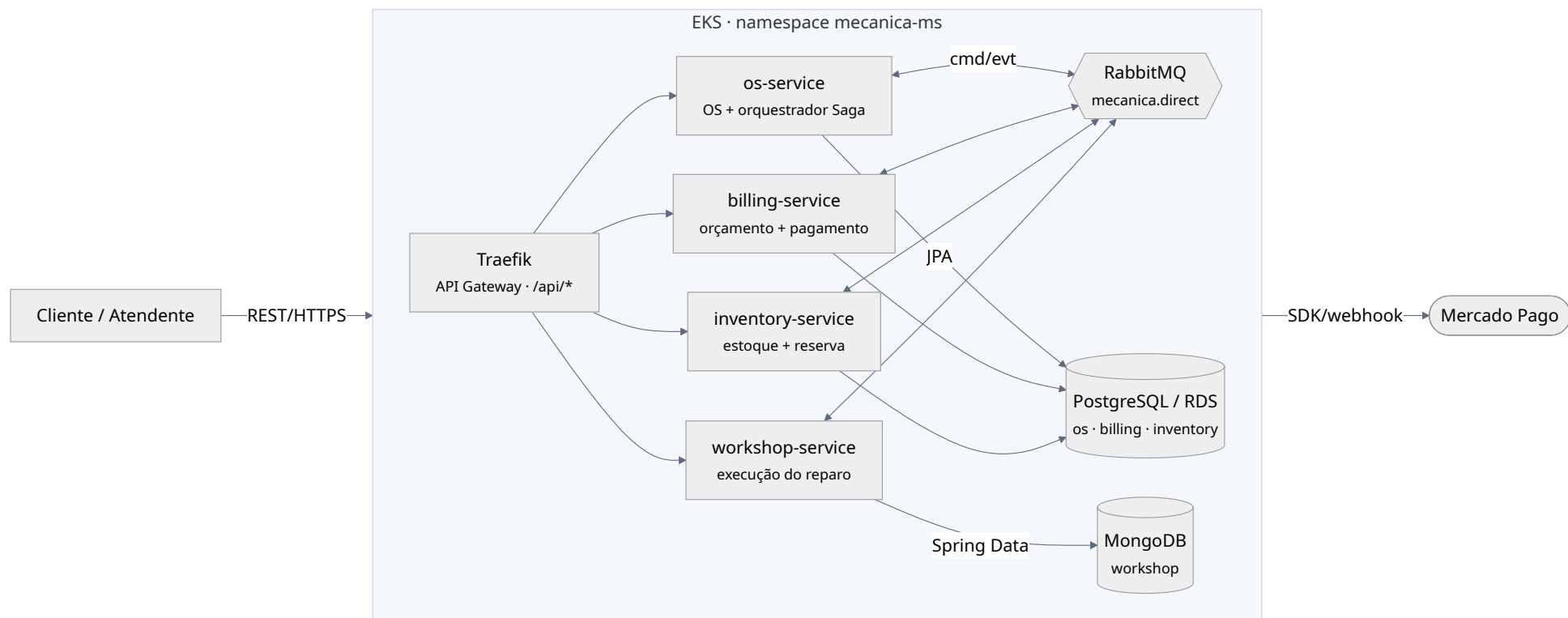
Java 21 · Spring Boot

Kubernetes / EKS

RabbitMQ

PostgreSQL + MongoDB

Quatro microsserviços por bounded context



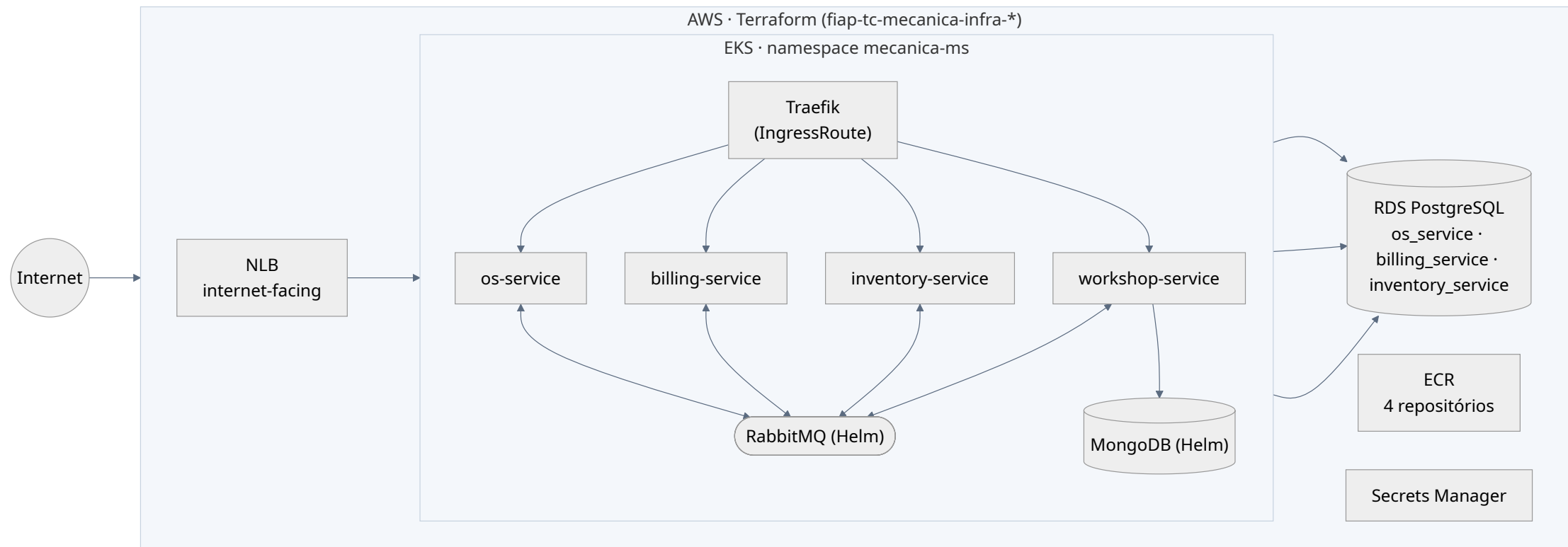
Princípios

- Um serviço por **bounded context** — repositório, CI/CD, infra e **banco próprios**
- Arquitetura **Hexagonal + DDD**; **shared-kernel** só compartilha Value Objects (tipos)

Comunicação

- **Síncrona/externa**: REST via Traefik (ponto único de entrada)
- **Assíncrona/interna**: RabbitMQ para a Saga (comandos + eventos)

Infraestrutura como código na AWS (EKS)



Provisionamento

VPC, EKS, ECR, RDS e Secrets Manager via **Terraform**. Traefik/MongoDB/RabbitMQ via **Helm**.

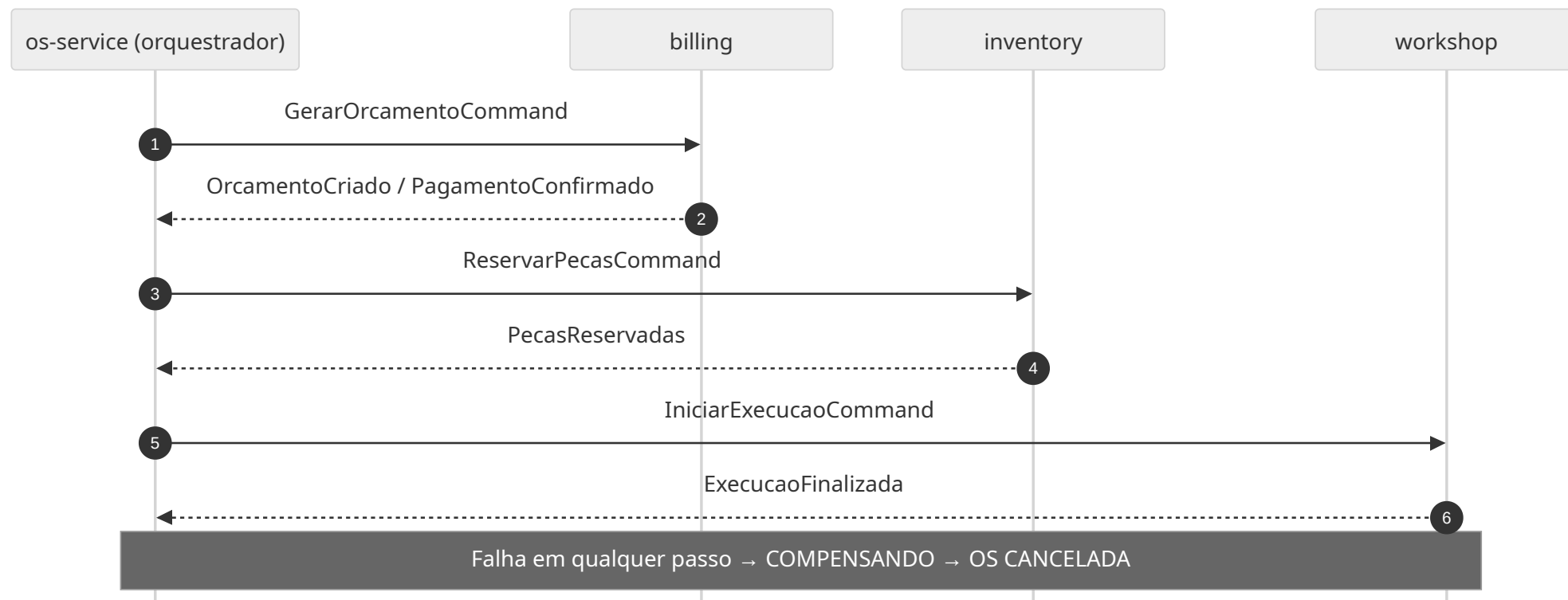
Entrega

CI (build/test/JaCoCo/Sonar) e CD (build → ECR → kubectl set image) por serviço.

Qualidade

Cobertura ≥ 80% (JaCoCo), BDD e **SonarCloud** Quality Gate por repositório.

Saga orquestrada via RabbitMQ



Por que orquestração

- **os-service** é o orquestrador (OsSagaCoordinator)
- Estado **persistido** (saga_state) → saga auditável e retomável
- Fluxo e compensações **explícitos e centralizados**

Mecanismo

- Exchange direct mecanica.direct: **comandos** (→ serviços) e **eventos** (→ orquestrador)
- **Idempotência** no consumo tolera reentrega do broker
- Rejeitados: coreografia (fluxo disperso) e 2PC/XA (bloqueante)

Poliglota + um banco por serviço

MongoDB no workshop-service (NoSQL)

- Execução do reparo = **agregado auto-contido**, estrutura flexível e aninhada (checklist, apontamentos, histórico)
- Leitura/escrita por `os_id`, **sem joins**
- Coleções execucoes, mecanicos, processed_commands
- Alternativas rejeitadas: JSONB no Postgres, DynamoDB

PostgreSQL nos demais (SQL)

- `os / billing / inventory`: dados **relacionais e transacionais**
- Requisito "próprio banco": **3 bancos lógicos isolados** numa instância RDS única (barato no AWS Academy)
- Bancos criados por **Job idempotente in-cluster** (RDS privado, sem exposição pública)
- Trocável por 3 instâncias sem alterar os serviços

Decisões arquiteturais (ADRs)

#	DECISÃO	ESCOLHA	PRINCIPAL TRADE-OFF
0001	Arquitetura	4 microsserviços (bounded contexts) + Hexagonal/DDD	Independência × complexidade operacional
0002	Transação distribuída	Saga orquestrada (RabbitMQ, estado persistido)	Fluxo explícito × consistência eventual
0003	Persistência do workshop	MongoDB (documento)	Aderência ao modelo × mais um motor
0004	Bancos SQL	1 RDS · 3 bancos lógicos · Job de bootstrap	Custo/simplicidade × isolamento lógico (não físico)

Stack

Java 21 · Spring Boot · RabbitMQ · PostgreSQL
· MongoDB · Traefik · Docker · Kubernetes

Infra & Entrega

Terraform (VPC/EKS/ECR/RDS) · Helm · GitHub
Actions (CI/CD) · AWS Academy

Qualidade

JaCoCo ≥ 80% · BDD · SonarCloud (org
mecanica-fiap-tc)